# A Distributed Multi-Agent AI Platform for Scalable Forensic Audit and Compliance

Podichetty Samiksha, Chandana Devanaboyina and Medapally Shiva Reddy
Department of Information Technology, Chaitanya Bharathi Institute of Technology, Hyderabad, India
Email: samiksha10.1b@gmail.com, chandanad3884@gmail.com

*Abstract*—Procurement audit tests every invoice against its order, receipt, vendor and approval records, and justifies each exception against a written policy. Large language models (LLMs) can read such policies, yet they detect errors while rarely citing the governing rule, and policy thresholds drift silently out of the prompts and code meant to enforce them. We present a distributed multi-agent platform for forensic audit whose central contribution is a *policy compiler*: the policy document is parsed into a typed clause registry that twelve deterministic controls execute before any LLM is invoked, so that no threshold resides in code or prompts and no model can retract a decidable finding. We formalise threshold drift and show that an auditor with frozen thresholds errs on exactly the displaced decisions, with a polarity fixed by the direction of the change. On synthetic ledgers of 40,000 invoices, the compiler recovered 196 of 196 thresholds from fourteen documents and 280 of 280 from twenty held-out ones; the controls made no wrong decision in 1,360,000 audits, whereas frozen thresholds misdecided up to 30.7% of invoices. On three benchmark ledgers the controls reached an F1 of 0.833 at a precision of 1.000, against 0.536 and 0.498 for an isolation forest and a one-class SVM, at under 20 $\mu$s per invoice. On a 34-invoice sample, agents using a local 7B model found 12 of 20 judgement anomalies, but only 0.375 of their findings named the violated rule. All data are synthetic.

*Index Terms*—forensic audit, multi-agent systems, large language models, policy as data, deterministic controls, threshold drift, procure-to-pay, explainability.

## I. Introduction

An auditor who tests procurement spend must establish, for every invoice, that it corresponds to an approved purchase order (PO), that the goods or services were received (goods receipt note, GRN), that the vendor is legitimate and tax-compliant, and that the payment was approved within the approver's delegated limit. The evidence is scattered across records, and what counts as an exception is fixed by an organisation-specific policy. Continuous auditing proposes to test every transaction rather than a sample [1].

LLMs are a natural candidate, because the policy is written in prose. AuditBench [2] delineates their limits: given transaction histories, LLMs identify errors, but explain them poorly and rarely cite the relevant standard. Systems of cooperating, specialised LLM agents are increasingly proposed for financial tasks [3]. Building such a pipeline exposed a second, structural problem, which we call the *policy–implementation gap*: tolerances and limits were stated in prose, one of them was duplicated as a literal in an agent prompt, and whether a rule was checked at all depended on an agent electing to call a tool. Every revision of the policy demanded a manual edit of the system, with no evidence that the edit matched the document.

This paper presents a distributed multi-agent platform for forensic procurement audit. *Forensic* means that every decision is reconstructible: each finding carries its rule, its evidence and the page of the policy it rests on, and every stage appends to an audit trail. *Distributed* means that the platform is a set of separable services communicating over network interfaces. We retain the base pipeline of [2, 3] and close the gap with one feature. Our contributions are:

1. a *policy compiler* that turns the policy document into an executable clause registry, with explicit refusal semantics for documents it cannot execute (Section V-B);
2. a controls-first architecture with a *non-interference* guarantee: no LLM stage can retract or weaken a deterministic finding (Proposition 1);
3. a formal account of *threshold drift*: the error of a frozen auditor equals the decision displacement of the policy change, and its polarity is fixed by the direction of the change (Propositions 2 and 3), confirmed on 34 policy variants;
4. a comparison with two classical anomaly detectors and an end-to-end evaluation of the LLM stages, with an open, reproducible artefact.

## II. Base Papers

### A. AuditBench

Wang et al. [2] pair financial tables with synthetic transactions into which errors are injected, and evaluate LLMs on financial-statement audit. Their finding that LLMs detect errors but explain them and cite the governing standard poorly is the reason our findings must name the rule they rely on. We adopt their construction of evaluation data with injected, labelled errors, and their requirement that a finding cite its rule. AuditBench measures models; it does not propose a system that removes the weakness, nor does it address procurement documents checked against an organisation's own policy.

### B. LLM-Based Multi-Agent Systems in Finance

Liu et al. [3] survey LLM-based multi-agent systems for financial markets and argue that agents which are heterogeneous, autonomous and boundedly rational suit markets better than traditional models. We adopt the architecture of specialist agents with their own roles and views of the data, whose outputs are combined. The survey concerns market simulation rather than audit, and names regulatory traceability of agent decisions as an open challenge, which an audit system must answer concretely.

## III. Related Work

**Anomaly detection.** Anomaly detection ranks records by how unusual they are [4]; statistical fraud detection has long been reviewed [5]. Isolation forests [6] and one-class support vector machines [7] are standard unsupervised choices,

and we use both as baselines. A detector returns a score, not a rule: an unusual record need not violate policy, a violation (an invoice approved above a delegated limit) need not look unusual, and a policy change requires retraining.

**LLMs in audit and compliance.** LLMs have matched financial text against audit requirements [8] and verified the regulatory compliance of financial reports [9]. Generated text can assert facts the input does not support [10], which an audit cannot accept. Retrieval-augmented generation [11] grounds answers in retrieved passages, as our copilot does, but grounding alone does not make a rule citation correct.

**Multi-agent LLM systems.** Surveys of LLM-based agents are given in [12]; debate between agents improves factual accuracy [13]; review of uncertain cases by a human remains the accepted safeguard for high-stakes decisions [14].

**Research gap.** Prior work either detects anomalies without naming a rule or lets an LLM reason over the policy text. To our knowledge, no published work lets the policy document itself configure executable audit controls, so that the thresholds a system applies are provably those the document states.

## IV. Problem Formulation

A ledger is a set $\mathcal{L} = \{x_1, \dots, x_N\}$ of transactions. A transaction $x$ is an invoice together with its PO, its GRN, the vendor master record, the approval-log entries and the vendor's other invoices.

**Definition 1** (Policy)**.** *A policy version is a tuple $P = (K, s, \kappa, \theta)$: a set $K$ of rule identifiers; a severity $s : K \to \{\textit{critical}, \textit{high}, \textit{medium}, \textit{low}\}$; a kind $\kappa : K \to \{\textit{control}, \textit{judgement}\}$, stating whether a rule is decidable by comparing fields; and parameters $\theta = (\theta_k)_{k \in K}$. We write $K_c = \{k : \kappa(k) = \textit{control}\}$.*

Our reference policy (version 2.5) has $|K| = 16$ rules, $|K_c| = 12$, and 14 parameter values. Let $V_P(x) \subseteq K$ be the rules that $x$ violates under $P$, computed by an independent validator, and $V_P^{\mathrm{c}}(x) = V_P(x) \cap K_c$. An auditor $\mathcal{A}$ cites a set $C_{\mathcal{A}}(x)$ of rules.

**Definition 2** (Adherence and displacement)**.** *The adherence of $\mathcal{A}$ to $P$ is*

$$\mathrm{A}_P(\mathcal{A}) = \frac{1}{N} \sum_{x \in \mathcal{L}} \mathbf{1}\big[C_{\mathcal{A}}(x) = V_P^{\mathrm{c}}(x)\big]. \tag{1}$$

*The decision displacement of a change from $P_0$ to $P$ is $\Delta(P_0, P) = \{x \in \mathcal{L} : V_P^{\mathrm{c}}(x) \neq V_{P_0}^{\mathrm{c}}(x)\}$.*

The *policy-driven* auditor executes the controls under parameters $\hat{\theta} = E(D)$ extracted from the document $D$ of $P$; the *frozen* auditor executes the same controls under $\theta_0$ of $P_0$, which models a system whose thresholds are written into prompts and code until someone edits them.

## V. Proposed System

### A. Architecture

The platform comprises audit workers, a policy service, a relational data store (SQLite or PostgreSQL), a model endpoint with an OpenAI-compatible interface, and a REST API with a web application for reviewers (Fig. 1). One invoice is one unit of work: its transaction carries every record it needs, so workers require no coordination. The pipeline is a LangGraph [15] state graph: evidence extraction, deterministic controls, three specialist agents in parallel (math and approvals; tax and identity; vendor intelligence), a challenger, a confidence stage, risk scoring and routing, and human review with an audit pack. Each agent sees only the records its rules need, and a finding that cites a rule outside its remit is discarded by code. Results are appended, never replaced.

**Algorithm 1** Policy compilation with refusal

**Input:** policy PDF $D$; reference policy $P_{\mathrm{ref}} = (K, s, \kappa, \theta_{\mathrm{ref}})$
**Output:** registry $\hat{P}$ or a list $\Pi$ of problems

$\Lambda \leftarrow$ text lines of $D$ with font size, position, page
$\mathcal{R} \leftarrow$ rules segmented from $\Lambda$ by layout; $\Pi \leftarrow \emptyset$
**for all** $k \in K$ **do**
  **if** $k \notin \mathcal{R}$ **then** $\Pi \leftarrow \Pi \cup \{$"rule $k$ absent"$\}$
  **else if** $k$ stated more than once **then** $\Pi \leftarrow \Pi \cup \{$"$k$ ambiguous"$\}$
  **else** $\hat{s}(k) \leftarrow$ severity stated in $\mathcal{R}_k$
    **for all** parameter $p$ of $k$ **do**
      $\hat{\theta}_{k,p} \leftarrow$ match pattern of $p$ in $\mathcal{R}_k$
      **if** no match or value not meaningful **then** $\Pi \leftarrow \Pi \cup \{(k, p)\}$
      **end if**
    **end for**
  **end if**
**end for**
add to $\Pi$: rules in $\mathcal{R} \setminus K$; no version; approval table missing a role
**if** $\Pi \neq \emptyset$ **then return** $\Pi$ ▷ refused; nothing stored
**end if**
**return** $\hat{P} = (K, \hat{s}, \kappa, \hat{\theta})$ with passages and pages

### B. Policy Compilation

The compiler (Algorithm 1) reads every text line of the PDF with its font size, position and page, and segments the text into numbered rules by layout rather than keyword: the modal font size is the body size, a line at least one point larger is a heading, and a line opening with "Rule ⟨number⟩ (⟨severity⟩)," starts a rule. One pattern per parameter is matched against the wording of the rule that states it; approval limits are read from the table of roles. No language model participates: a value is either found in its sentence or reported missing. The rule set $K$ and the kinds $\kappa$ come from a reference version, which records what the system can execute.

### C. Deterministic Controls

A control $c_k(x; \theta_k)$ is a pure function of a transaction and the parameters of its rule. It returns nothing or a finding $f = (k, s(k), E)$, where $E$ is the set of field values it rests on; when the result is stored, the finding is attached to the document and page of its rule. A control contains no constant of its own. For example, the amount-match control raises a finding iff

$$100\,|a_{\mathrm{inv}} - a_{\mathrm{po}}| > \hat{\theta}_{2.1.1}\, a_{\mathrm{po}}, \tag{2}$$

with amounts in integer paise, so that rounding cannot move a decision at the boundary. The approval-limit control takes the limit of the approving role from the registry, not from the approval log, because the log records what the approval system believed at the time. The controls run as $F_D(x) = \{c_k(x; \hat{\theta}_k) : k \in K_c\}$.

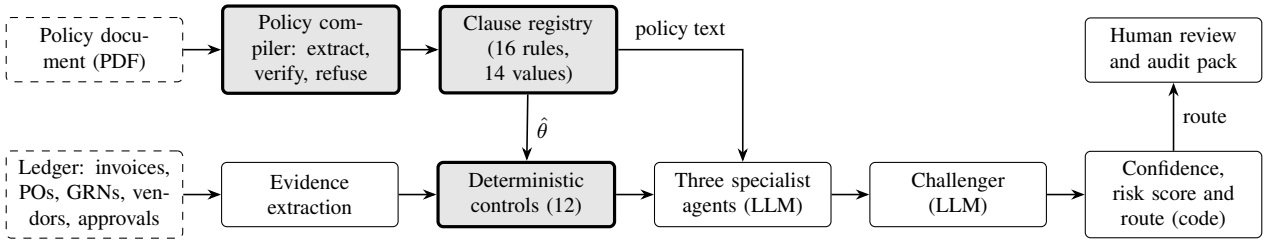



Fig. 1. The proposed system. Grey boxes with a thick border are the extra feature; white boxes the base pipeline; dashed boxes data. The policy document configures the controls, which run before any LLM is called; scoring and routing are code.

### D. Agentic Layer

The four judgement rules (receipt remarks, description match, near-duplicates, structuring across days) are assigned to three specialist agents whose remits follow the sections of the policy. Prompts contain no threshold and no rule number; the policy text is inserted at run time. The challenger, in the spirit of multi-agent debate [13], may remove agent findings that the evidence does not support. In configuration C5 each model finding $f$ receives a confidence

$$c(f) = \tfrac{1}{3}\big(g(f) + p(f) + a(f)\big), \tag{3}$$

where $g = 1$ if every evidence field exists in the transaction with the stated value, $p = 1$ if the cited rule exists in the active policy, and $a$ is the share of $m = 3$ re-runs at temperature 0.7 that cite the same rule; $c(f) = 1$ for a control finding.

### E. Risk Scoring and Routing

With weights $w = (30, 20, 10, 5)$ for critical, high, medium and low findings, the count-based and confidence-weighted scores are

$$R_0(x) = \min\Big(100, \textstyle\sum_{f \in F(x)} w_{s(f)}\Big), \tag{4}$$

$$R_c(x) = \min\Big(100, \textstyle\sum_{f \in F(x)} w_{s(f)}\, c(f)\Big), \tag{5}$$

each raised to at least 70 (50) by a critical (high) finding, for $R_c$ only by findings with $c(f) \geq \vartheta = 2/3$. An invoice is escalated if $R_c \geq \tau = 50$, sent to sampled review if $R_c < \tau \leq R_0$, and approved otherwise.

### F. Formal Properties

**Proposition 1** (Non-interference)**.** *In every configuration with controls, $F_D(x) \subseteq F(x)$ and each $f \in F_D(x)$ enters (4)–(5) with $c(f) = 1$. Hence $R_c(x) \geq R_0^{\mathrm{C1}}(x)$, and every invoice escalated by the controls alone remains escalated whatever the LLM stages output.*

*Sketch.* The challenger's action space is restricted to agent findings, and model findings add non-negative terms to (5); the floors triggered by control findings are unaffected. □

**Proposition 2** (Drift identity)**.** *If the controls are faithful under $P_0$, i.e. $C(x) = V_{P_0}^{\mathrm{c}}(x)$ for all $x$, then the frozen auditor satisfies $\mathrm{A}_P(\textit{frozen}) = 1 - |\Delta(P_0, P)|/N$.*

*Sketch.* The frozen auditor cites $V_{P_0}^{\mathrm{c}}(x)$, which equals $V_P^{\mathrm{c}}(x)$ iff $x \notin \Delta(P_0, P)$. □

**Proposition 3** (Error polarity)**.** *If a change tightens the policy, $V_{P_0}^{\mathrm{c}}(x) \subseteq V_P^{\mathrm{c}}(x)$ for all $x$, every error of the frozen auditor is an omission (a missed exception); if it loosens the policy, $V_{P_0}^{\mathrm{c}}(x) \supseteq V_P^{\mathrm{c}}(x)$, every error is a commission (an unsupported finding).*

*Sketch.* Errors are $V_P^{\mathrm{c}} \setminus V_{P_0}^{\mathrm{c}}$ (omissions) and $V_{P_0}^{\mathrm{c}} \setminus V_P^{\mathrm{c}}$ (commissions); one of the two is empty under each inclusion. For threshold rules the inclusion follows from the order of the parameter: a smaller tolerance, a lower daily count or approval limit, a superset of non-business days or blocked statuses, or a divisor of the round-amount multiple tightens. □

Propositions 2 and 3 make the frozen system's error a property of the ledger and the change, not of the implementation: Section VII measures it.

## VI. Experimental Setup

**Data.** All data are synthetic (AuditIQ-Bench). Following AuditBench [2], anomalies are injected into synthetic transactions and labelled when injected; an independent validator, sharing no code with the controls, recomputes $V_P$ from the finished tables and rejects any ledger whose labels it does not reproduce. Three benchmark ledgers of 5,000 invoices hold 420 anomalous invoices (8.4%) in fourteen classes, ten rule-defined and four judgement classes, and 120 hard negatives: clean invoices built to resemble an anomaly. A 20,000-invoice ledger holds 420 anomalous invoices of the same composition (2.1%). Five ledgers of 40,000 invoices in total carry the policy experiments.

**Policies.** From version 2.5 we derive fourteen variants (eleven change one threshold, three change several), twenty further variants drawn at random with a fixed seed from value ranges never used to build or test the compiler, and sixteen edited copies of the reference document that test refusal. Each is rendered as a PDF, the only input the system receives.

**Baselines.** An isolation forest [6] (300 trees, fitted without labels on all invoices) and a one-class SVM [7] (RBF kernel, $\nu = 0.1$, fitted on the clean invoices of the training folds), from scikit-learn [16], on 21 generic features that encode no policy threshold. Thresholds are set by five-fold cross-fitting, stratified by class: each fold is decided with the threshold that maximises F1 on the other four, so no invoice is decided with a threshold tuned on its own label. As a detector names no rule, an invoice counts as detected when reported at all.

**LLM stages.** Qwen2.5-7B [17] served locally on a laptop CPU, temperature 0. Configurations: C1, controls alone; C2, one prompt over the whole policy, no controls; C3, controls and three agents; C4, C3 with the challenger; C5, C4 with confidence. Prompts were tuned only on a development ledger and frozen before evaluation (one prompt hash

TABLE I. ADHERENCE TO THE POLICY IN FORCE ON 40,000 INVOICES PER POLICY

| Policy variant | $\lvert\Delta\rvert$ | Adherence driven | Adherence frozen | Omiss. (frozen) | Commiss. (frozen) |
|---|---|---|---|---|---|
| Tolerance 0.5% | 62 | 1.000 | 0.998 | 62 | 0 |
| Daily count 2 | 429 | 1.000 | 0.989 | 429 | 0 |
| Manager limit 50k | 7,677 | 1.000 | 0.808 | 7,677 | 0 |
| Sr. Manager limit 250k | 4,259 | 1.000 | 0.894 | 4,259 | 0 |
| Strict policy | 12,274 | 1.000 | 0.693 | 12,427 | 0 |
| Lenient policy | 380 | 1.000 | 0.991 | 0 | 380 |
| Version 2.6 | 302 | 1.000 | 0.992 | 0 | 302 |

Omissions and commissions count rules; $\lvert\Delta\rvert$ counts invoices.

across all runs). We evaluate on 20 invoices of the development ledger (one per anomaly class, one per hard-negative type, one ordinary) and on a 34-invoice sample of benchmark ledger 101 (five per judgement class, two per hard-negative type, four ordinary).

**Statistics.** 95% percentile bootstrap intervals over invoices (1,000 resamples); McNemar's exact test on paired per-invoice correctness, Holm-adjusted.

## VII. RESULTS

### A. Compilation Fidelity

The compiler accepted all fourteen variants and recovered 196 of 196 parameter values and 224 of 224 rules word for word, in 0.01–0.09 s per document. On the twenty held-out variants it recovered 280 of 280 values and 320 of 320 rules. Of the sixteen edited documents, all seven that preserve every value were read correctly and all seven that make the document unexecutable were refused with a reason. The two ambiguous documents, a rule stated twice with two values and an approval table lacking a role, were adopted in a first run; both checks were added with regression tests, after which all sixteen were handled as required.

### B. Adherence and Threshold Drift

The policy-driven controls decided every invoice as the policy in force requires: no wrong decision in 560,000 audits over the fourteen variants and 800,000 over the twenty held-out variants (Table I). Under version 2.5 the controls and the validator agree on 40,000 of 40,000 invoices, so Proposition 2 applies; indeed the frozen auditor's errors equal the decision displacement in all 34 variants. Displacement ranges from none to 12,274 invoices (30.7%) over the fourteen variants and up to 7,953 (19.9%, median 79) over the held-out ones. Polarity behaves as Proposition 3 predicts: each of the fourteen variants produces errors of a single polarity. The five tightening variants cause 24,854 missed exceptions and 24,177 missed escalations, a silent failure; the eight loosening variants cause 1,069 unsupported findings and 913 needless escalations, a cost in reviewer time. Each of the five held-out variants with both polarities combines a tightening and a loosening component, such as replacing Sunday by Friday as a non-business day.

### C. Detection Against Classical Detectors

Table II compares the controls with the detectors. The controls attain the highest recall and F1 at a precision of 1.000 and raise no finding on any clean invoice, hard negatives included, whereas the detectors flag 36% to 56% of the hard

TABLE II. INVOICE-LEVEL DETECTION: MEAN OF THREE BENCHMARK LEDGERS (5,000 INVOICES, 8.4% ANOMALOUS)

| Measure | Isol. forest | One-class SVM | Controls |
|---|---|---|---|
| Recall | 0.577 | 0.464 | 0.714 |
| Precision | 0.501 | 0.543 | 1.000 |
| F1 | 0.536 | 0.498 | 0.833 |
| FPR, hard negatives | 0.403 | 0.481 | 0.000 |
| FPR, ordinary | 0.044 | 0.025 | 0.000 |
| F1 at 2.1% (20,000 inv.) | 0.211 | 0.429 | 0.833 |

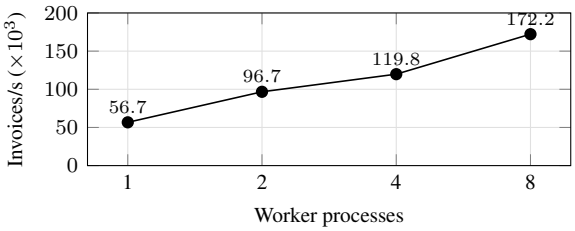



Fig. 2. Throughput of the control stage on one laptop (median of three runs).

negatives across the benchmark ledgers. McNemar's test favours the controls over each detector on every benchmark ledger (Holm-adjusted $p < 10^{-50}$). At the realistic anomaly rate of the 20,000-invoice ledger, the isolation forest falls to an F1 of 0.211 and the one-class SVM to 0.429, while the controls keep 0.833. The controls' recall of 1.000 on the ten rule-defined classes is by construction, since they implement the definitions by which those anomalies were injected; the detectors' behaviour is not. Strikingly, a single binary feature (the invoice GSTIN differs from the registered one) separates the two GSTIN classes from every ordinary invoice (on ledger 101, 30 of 30 invoices per class against 0 of 4,460), yet the one-class SVM reports on average only 3.3% of them; both detectors find the near-duplicates (J1) that the controls leave to the agents, but name no rule.

### D. Throughput

The control stage costs 17.8–19.4 $\mu$s per invoice in one process, independently of ledger size between 5,000 and 40,000 invoices, so its cost is linear in the ledger. Because invoices are independent units of work, the stage parallelises without coordination: on 300,000 invoice audits, throughput rose from 56,660 invoices per second with one worker to 172,226 with eight (speed-up 3.04 on 2 performance and 8 efficiency cores), every configuration producing the same 16,840 findings (Fig. 2). With every result, finding and trail entry stored, one process audits 20,000 invoices in under a minute.

### E. The LLM Stages

Table III summarises the 34-invoice sample. The controls alone find none of the 20 judgement anomalies, by design. The agents of C3 found 12: all five near-duplicates, four of five goods-receipt remarks, two of five cross-day structuring cases and one of five description mismatches; yet only 0.375 of their 32 findings named a violated rule. The single prompt of C2 found 4, flagged seven of ten hard negatives and all four ordinary invoices, and named a violated rule in 0.031 of its 128 findings. The challenger of C4 halved the false

TABLE III. LLM CONFIGURATIONS ON THE 34-INVOICE SAMPLE (QWEN2.5-7B)

| Measure | C2 | C3 | C4 | C5 |
|---|---|---|---|---|
| Judgement anomalies found (of 20) | 4 | 12 | 8 | 8 |
| Hard negatives flagged (of 10) | 7 | 8 | 4 | 4 |
| Ordinary invoices flagged (of 4) | 4 | 3 | 2 | 2 |
| Findings naming a violated rule | 0.031 | 0.375 | 0.471 | 0.471 |
| Sent to sampled review | – | – | – | 4 |
| Model calls per invoice | 1.21 | 3.00 | 3.74 | 5.24 |

alarms on hard negatives (eight to four) and raised the share of findings naming a violated rule to 0.471, but removed four of the five correct near-duplicate findings. C5 sent four invoices to sampled review, three of them true anomalies. Agreement over re-runs did not separate right from wrong: 6 of 9 wrong and 5 of 8 right model findings kept by C5 had full agreement. On the 20-invoice pilot, every rule-defined class was decided correctly with the controls first, and, as Proposition 1 requires, all ten control findings and all eight control escalations survived C3–C5 unchanged. No difference between configurations is significant (C3 against C2, $p = 0.277$).

## VIII. DISCUSSION

The adherence of 1.000 certifies the mechanism, not the rules: the controls and the validator are two implementations of the same written rules, and their agreement shows that each document was compiled correctly and that no control harbours a threshold of its own. The substantive quantities are those of the frozen auditor. By Proposition 2 its error is the displacement of the change, and ordinary revisions of a procurement policy displace between none and almost a third of the decisions on these ledgers; by Proposition 3, a frozen system fails silently under every tightening.

For the twelve decidable rules, the citation failure that AuditBench reports cannot arise: a finding is bound to its rule by the control that implements it. For the four judgement rules, the agents' low share of findings naming a violated rule reproduces that failure, and neither the challenger nor agreement-based confidence removes it; this is why those rules remain under human review. The architecture thus does not solve LLM unreliability; it *confines* it, moving every decidable rule out of the model's reach (Proposition 1) and leaving the model's errors where a reviewer can see them.

## IX. LIMITATIONS

**Synthetic data.** All ledgers are synthetic; the counts describe how these ledgers react to these changes, not the books of an organisation. **Circularity.** The controls and the ground truth implement the same written rules; perfect scores on the rule-defined classes verify the implementation, not that the rules catch real fraud. **LLM evaluation.** The LLM stages were measured on 20 and 34 invoices with one 7B model on a laptop CPU; the samples are too small to separate configurations, and accuracy on whole benchmark ledgers is unmeasured. **Compiler scope.** Pattern extraction reads one template's wording and refuses other documents with reasons; a model-assisted path for documents in their own wording proposed 80 of 84 values correctly on six short documents, but two wrong values passed its code check, so a named person must confirm before adoption. **Deployment.** All services ran on one laptop; a multi-machine deployment has not been measured.

## X. CONCLUSION AND FUTURE WORK

We presented a distributed multi-agent platform for forensic procurement audit in which the organisation's policy document is compiled into the thresholds of deterministic controls that run before, and cannot be overridden by, any LLM. On synthetic data the compiler recovered every threshold of fourteen hand-written and twenty held-out documents, the controls made no wrong decision in 1,360,000 invoice audits, and a frozen auditor misdecided up to 30.7% of invoices, exactly as the drift identity predicts. The controls reached an F1 of 0.833 without a false alarm on a clean invoice, against 0.536 and 0.498 for classical detectors, and audited 172,226 invoices per second on one laptop. Future work will evaluate the LLM stages on whole benchmark ledgers with a larger model, deploy the services across machines, and evaluate the platform on real, anonymised procurement data.

## DATA AND CODE AVAILABILITY

The platform, the datasets, the policy documents and the scripts that produce every number are available at `https://github.com/Chandana909/major.project`.

## REFERENCES


[1] M. A. Vasarhelyi and F. B. Halper, "The continuous audit of online systems," *Auditing: J. Pract. Theory*, vol. 10, no. 1, pp. 110–125, 1991, doi: 10.2308/ajpt-9703192366.

[2] R. Wang, J. Liu, W. Zhao, S. Li, and D. Zhang, "AuditBench: A benchmark for large language models in financial statement auditing," in *AI for Research and Scalable, Efficient Systems*, CCIS. Singapore: Springer, 2025, pp. 59–81, doi: 10.1007/978-981-96-8912-5_3.

[3] Q. Liu, L. Yao, Z. Wang, Y. Yang, Y. Tang, D. Cheng, and C. Jiang, "Large language model-based multi-agent systems for financial markets simulation: A survey," *Sci. China Inf. Sci.*, vol. 69, no. 7, Art. no. 171201, 2026, doi: 10.1007/s11432-026-4986-x.

[4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009, doi: 10.1145/1541880.1541882.

[5] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002, doi: 10.1214/ss/1042727940.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[7] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001, doi: 10.1162/089976601750264965.

[8] L. Hillebrand *et al.*, "Improving zero-shot text matching for financial auditing with large language models," in *Proc. ACM Symp. Document Eng.*, 2023, pp. 1–4, doi: 10.1145/3573128.3609344.

[9] A. Berger *et al.*, "Towards automated regulatory compliance verification in financial auditing with large language models," in *Proc. IEEE Int. Conf. Big Data*, 2023, pp. 4626–4635, doi: 10.1109/BigData59044.2023.10386518.

[10] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023, doi: 10.1145/3571730.

[11] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[12] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proc. 33rd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2024, pp. 8048–8057, doi: 10.24963/ijcai.2024/890.

[13] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multi-agent debate," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 235, 2024.

[14] E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, and Á. Fernández-Leal, "Human-in-the-loop machine learning: A state of the art," *Artif. Intell. Rev.*, vol. 56, no. 4, pp. 3005–3054, 2023, doi: 10.1007/s10462-022-10246-w.

[15] LangChain, "LangGraph," software. [Online]. Available: `https://github.com/langchain-ai/langgraph`

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[17] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.